

Índice

Cazador de Recompensas – (🔗 decorator-profugos)	1
¿Por qué nos dan la interfaz IProfugo desde el comienzo?	1
Construcción técnica del Decorator	1
Cómo resuelve cada entrenamiento su responsabilidad	2
Diagrama de clases decorator	4
Conclusión: la gracia del patrón	5
Template Method en Cazador	5
¿Y si usáramos Strategy?	5
Patrón Singleton en Agencia	5
Parte III — Reportería con Agencia y flatMap	6
¿Por qué flatMap para obtener todos los prófugos capturados?	6
¿Cómo funciona max() con Comparator ?	7
Comparator.comparing en getProfugoMasHabil	8

Cazador de Recompensas – (🔗 decorator-profugos)

¿Por qué nos dan la interfaz **IProfugo** desde el comienzo?

Es un indicio fuerte de que vamos a usar Decorator. Sin interfaz, el patrón no es posible, necesitamos un tipo común (**IProfugo**) para que tanto el componente concreto (**Profugo**) como los decoradores sean intercambiables. Si el enunciado fuese distinto (ej: solo un **Profugo** concreto sin evolución), bastaba una clase sola y no hacía falta la interfaz. Al darla desde el principio, nos están diciendo «vas a necesitar polimorfismo sobre **IProfugo**», que es exactamente lo que pide el Decorator cuando los prófugos evolucionan apilando entrenamientos.

Construcción técnica del Decorator

Tenemos cuatro actores:

Clase	Rol
IProfugo (Interfaz)	<i>Componente.</i> Define todas las operaciones: <code>getInocencia()</code> , <code>getHabilidad()</code> , <code>esNervioso()</code> , <code>volverseNervioso()</code> , <code>dejarDeEstarNervioso()</code> , <code>reducirHabilidad()</code> , <code>disminuirInocencia()</code>

Clase	Rol
Profugo	<i>Componente Concreto.</i> Implementa IProfugo con estado real. Es el objeto base sin entrenamiento.
ProfugoDecorator (abstracta)	<i>Decorator Base.</i> Implementa IProfugo y tiene una referencia a un IProfugo envuelto (el «wrappee»). Su rol es delegar todos los métodos al wrappee.
ArtesMarciales, EntrenamientoElite, ProteccionLegal	<i>Decoradores Concretos.</i> Extienden ProfugoDecorator y overridean solo los métodos que modifican.

ProfugoDecorator es la clave: no repite lógica, solo pasa el mensaje al wrappee. En cada override de los concretos hacemos algo antes o después de llamar a `super.metodo()`, o evitamos la llamada si queremos bloquear el comportamiento.

El armado se hace por composición en el constructor:

```
1 IProfugo p = new ProteccionLegal(
2         new ArtesMarciales(
3         new Profugo(...)));
```

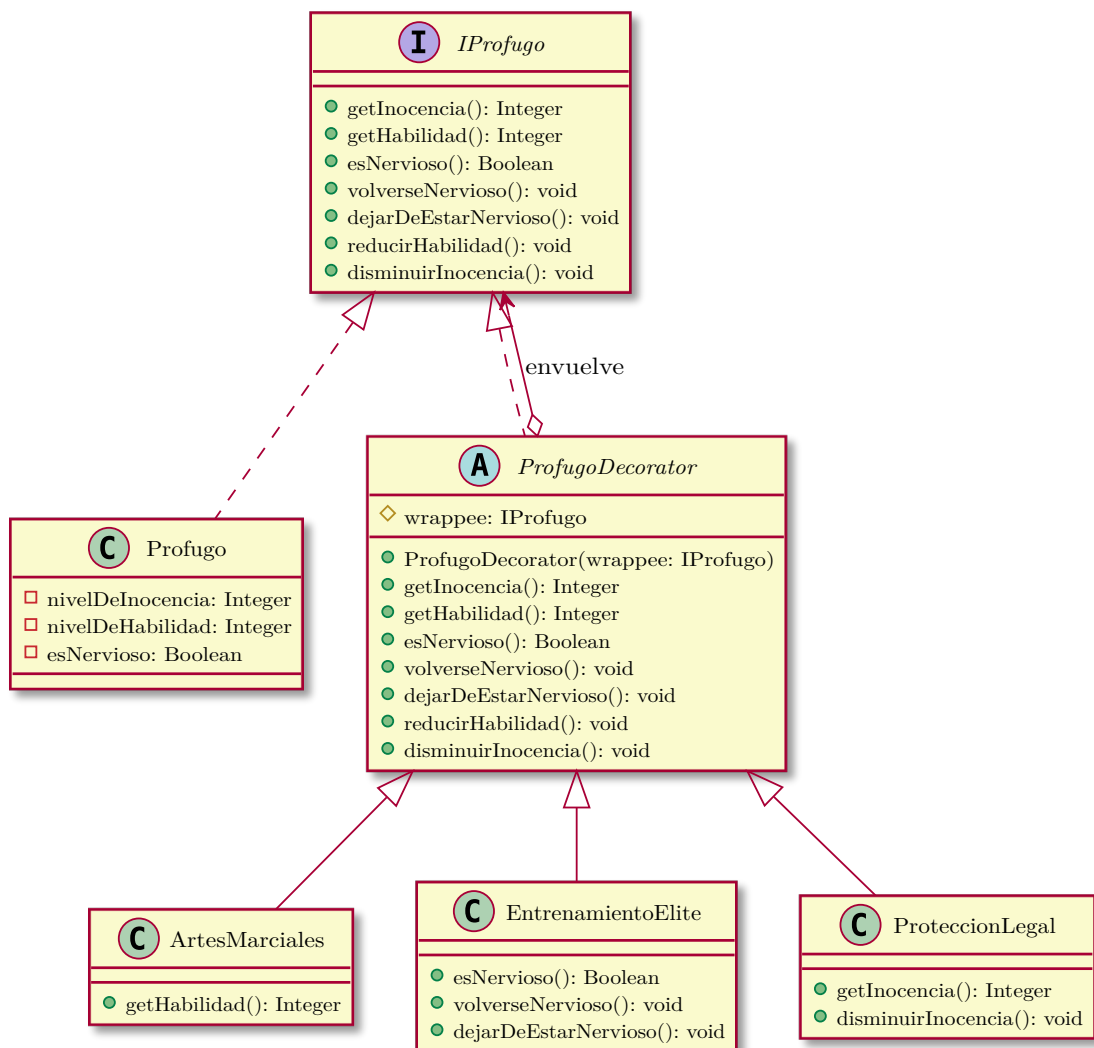
El orden importa: el decorador más externo envuelve al siguiente, que envuelve al siguiente, etc. `p` es un **IProfugo** y el código cliente no sabe ni le importa cuántas capas tiene.

Cómo resuelve cada entrenamiento su responsabilidad

Decorator	Método overrideado	Comportamiento
ArtesMarciales	<code>getHabilidad()</code>	Obtiene el valor del wrappee (<code>wrappee.getHabilidad()</code>), lo duplica, lo capsula a 100 y lo devuelve. El resto de métodos se delegan sin cambios.
EntrenamientoElite	<code>esNervioso()</code>	Siempre devuelve <code>false</code> .

Decorador	Método overrideado	Comportamiento
EntrenamientoElite	<code>volverseNervioso()</code>	No hace nada (bloquea el efecto).
EntrenamientoElite	<code>dejarDeEstarNervioso()</code>	No está overridden, delega al wrappee. Al estar pisado <code>esNervioso()</code> en false, es irrelevante.
ProteccionLegal	<code>getInocencia()</code>	Devuelve <code>Math.max(40, wrappee.getInocencia())</code> .
ProteccionLegal	<code>disminuirInocencia()</code>	Solo delega al wrappee si su inocencia actual es mayor a 40. Si está en 40 o menos, no hace nada.

Diagrama de clases decorator



Leyenda de flechas: $\cdots\rightarrow$ — implements (realización de interfaz), $\rightarrow$ — extends (herencia de clase), $\diamond \rightarrow$ — agregación (envuelve, tiene-un).

ProfugoDecorator tiene solo dos cosas que la diferencian de IProfugo:

1. **El atributo wrappee: IProfugo** — la referencia al objeto envuelto.
2. **Delegación pura** — implementa todos los métodos de IProfugo llamando al mismo método del wrappee, sin agregar lógica.

Los decoradores concretos extienden ProfugoDecorator y **overlinean solo los métodos que modifican**, llamando a `super.metodo()` (que delega al wrappee) y alterando el resultado. Sin el wrappee no hay decoración posible. Es el núcleo del patrón.

ProfugoDecorator implementa **todos** los métodos de IProfugo porque es un **wrapper completo**. Desde afuera, un ProfugoDecorator se comporta exactamente como un IProfugo — el cliente (cualquier código que use un IProfugo) no sabe si está hablando con un Profugo base, un decorador, o una pila de decoradores.

Conclusión: la gracia del patrón

Cada decorador agrega una responsabilidad única sin modificar la clase base, y se pueden combinar arbitrariamente sin explosión de subclases. La interfaz **IProfugo** es el pegamento que hace esto posible: sin ella, no podríamos apilar comportamientos de forma transparente.

Template Method en Cazador

Cazar e intimidar tienen una estructura fija con pasos variables. Template Method evita repetir la estructura en cada subclase.

- `cazar()` decide entre capturar o intimidar. La condición de captura tiene una parte general (`experiencia > inocencia`) y una parte específica (`doPuedeCazar()`) que cada cazador define distinto.
- `intimidar()` siempre baja la inocencia 2 unidades, y después aplica el efecto específico (`doIntimidar()`) de cada cazador.

Ventaja: si mañana aparece un nuevo tipo de cazador, solo define `doPuedeCazar()` y `doIntimidar()`. La estructura del proceso de captura no se toca. Si olvida implementarlos, no compila.

¿Y si usáramos Strategy?

Template Method y Strategy resuelven problemas parecidos pero con estructuras opuestas.

Template Method usa herencia: la superclase define el esqueleto del algoritmo y las subclases completan los pasos variables. La variación está «en la clase» – el cazador **es** de un tipo y actúa según su naturaleza.

Strategy usa composición: el algoritmo se extrae a una interfaz aparte y se inyecta desde afuera. La variación está «en un objeto externo» – el cazador **tiene** una estrategia que puede cambiarse en tiempo de ejecución.

En este proyecto, Template Method es más natural porque cada cazador tiene una identidad fija (Urbano, Rural, Sigiloso) y no necesita cambiar su forma de cazar después de crearse. Con Strategy ganarías flexibilidad para cambiar la condición de captura dinámicamente, pero perderías la relación directa entre «tipo de cazador» y «forma de cazar» que pide el enunciado.

Patrón Singleton en Agencia

La Agencia usa Singleton porque existe una única agencia coordinando a todos los cazadores, consistente con el README.

```
1  public class Agencia {
2
3      // Atributo estático privado: guarda la única instancia de la
      // clase.
4      // static: el campo existe sin necesidad de una instancia previa.
```

```

5      //      Sin static haría falta new Agencia() para que exista,
6      //      pero el constructor privado lo impide.
7      // final: la referencia no se puede reasignar. Sin final, alguien
8      //      podría pisar INSTANCIA con otra instancia y romper el
9      //      singleton.
10     private static final Agencia INSTANCIA = new Agencia();
11
12     // Constructor privado: evita que externos creen instancias con
13     // new
14     private Agencia() {}
15
16     // Método estático público: punto de acceso global a la única
17     // instancia.
18     // static: al pertenecer a la clase y no a una instancia, puede
19     // llamarse con Agencia.getInstance() desde cualquier
20     // parte.
21     public static Agencia getInstance() {
22         return INSTANCIA;
23     }
24 }

```

Los tres ingredientes del Singleton:

1. **Atributo estático privado** (`private static final`) — guarda la única instancia.
2. **Constructor privado** (`private Agencia()`) — nadie afuera puede hacer `new`.
3. **Método estático público** (`public static Agencia getInstance()`) — punto de acceso global.

Parte III — Reportería con Agencia y flatMap

Para consolidar las capturas usamos Agencia que centraliza todos los cazadores.

¿Por qué `flatMap` para obtener todos los prófugos capturados?

Cada cazador tiene su propio `Set<IProfugo>` `profugosCapturados`. Tenemos una lista de cazadores → una lista de listas de profugos.

Con `map` obtendríamos un `Stream<Set<IProfugo>>` (stream de sets), pero nosotros queremos un `Stream<IProfugo>` con todos los profugos aplanados en un solo flujo. Ahí entra `flatMap`:

```

1 public Set<IProfugo> getProfugos() {
2     return cazadores.stream()
3         .flatMap(c -> c.getProfugosCapturados().stream())
4         .collect(Collectors.toSet());
5 }

```

Visualmente:

```
1 cazadores = [cazador1,      cazador2,      cazador3]
2               |              |              |
3             {p1, p2}        {p3, p4}        {p5}
4
5 flatMap → [p1, p2, p3, p4, p5]
```

Sin `flatMap` tendríamos que anidar dos loops (cazadores + profugos). `flatMap` hace exactamente eso, pero funcional.

¿Cómo funciona `max()` con `Comparator`?

`max()` arranca con el primer elemento como «ganador». Después agarra cada elemento de la lista de a uno y le pregunta al comparador:

«Entre este nuevo (candidato) y el que va ganando (ganador), ¿cuál es más grande?»

El comparador devuelve un número:

- **Positivo** → el candidato es más grande → `max()` reemplaza al ganador (ganador = candidato).
- **Negativo** → el candidato es más chico → el ganador sigue igual.
- **Cero** → son iguales → el ganador sigue igual.

Al final, el que sobrevivió a todas las comparaciones es el resultado.

Ejemplo con lista [3, 7, 2, 9, 1]:

```
1 Arranca: ganador = null
2
3 Paso 1: agarra el 3
4     ganador es null → 3 es el nuevo ganador
5     ganador = 3
6
7 Paso 2: agarra el 7
8     comparador.compare(7, 3) → 7.compareTo(3)
9     → devuelve 1 (positivo, 7 > 3)
10    positivo → 7 es el nuevo ganador
11    ganador = 7
12
13 Paso 3: agarra el 2
14    comparador.compare(2, 7) → 2.compareTo(7)
15    → devuelve -1 (negativo, 2 < 7)
16    negativo → 7 sigue siendo ganador
17    ganador = 7
18
```

```

19 Paso 4: agarra el 9
20     comparador.compare(9, 7) → 9.compareTo(7)
21     → devuelve 1 (positivo, 9 > 7)
22     positivo → 9 es el nuevo ganador
23     ganador = 9
24
25 Paso 5: agarra el 1
26     comparador.compare(1, 9) → 1.compareTo(9)
27     → devuelve -1 (negativo, 1 < 9)
28     negativo → 9 sigue siendo ganador
29     ganador = 9
30
31 Resultado: 9

```

Cada vez que `max()` agarra un elemento, lo compara con el ganador actual usando la función que le pasaste. Esa función siempre recibe **dos** valores: el nuevo candidato y el que va ganando hasta ese momento.

Comparator.comparing en getProfugoMasHabil

En la Agencia:

```

1 public IProfugo getProfugoMasHabil() {
2     return getProfugos()
3         .stream()
4         .max(Comparator.comparing(IProfugo::getHabilidad))
5         .orElse(null);
6 }

```

`Comparator.comparing(IProfugo::getHabilidad)` internamente equivale a:

```

1 new Comparator<IProfugo>() {
2     @Override
3     public int compare(IProfugo a, IProfugo b) {
4         return a.getHabilidad().compareTo(b.getHabilidad());
5     }
6 }

```

`comparing` extrae el `Integer` de cada profugo usando `getHabilidad()` y luego usa el `compareTo` nativo de `Integer` para decidir cuál es más grande. Es `a.compareTo(b)` pero aplicado a las habilidades extraídas, no a los profugos directamente.